Mimir - A Control Plane Based on Responsibility Disaggregation

Antony Amaral
costa@telecom-paris.fr
Institute Polytechnic of Paris
Palaiseau, France

Abstract

Function as a Service (FAAS) platforms allow users to deploy stateless functions that are executed on demand in the cloud, without requiring infrastructure management. The efficiency of these platforms depends heavily on the design of their control planes, particularly the communication schemes used to coordinate function execution. In this paper, we introduce Mimir, a control plane architecture that delegates scheduling decisions to workers and simplifies the request path, contrasting it with existing solutions such as Dirigent. Mimir distributes autonomy among the Controller, Data Plane, and Workers, and employs a publisher/subscriber queue system for efficient state propagation.

We compare Mimir and Dirigent in both cold and warm start scenarios, focusing on how their communication structures affect end to end communication latency. The results show that Mimir's communication schema significantly reduces latency in cold start scenarios, while maintaining comparable performance in warm starts.

1 Introduction

Functions as a Service (FAAS) are a consolidated type of service available in many cloud providers, such as Azure Cloud, Google Cloud, and Amazon Web Services. The basic idea is that a user submits stateless function code, which is then executed on a cloud server without the user needing to manage or provision the underlying infrastructure. This model creates a rich environment for research because of the diversity of user code and the complex infrastructure required to run it. As a result, serverless function control planes have been improved over time, with many different strategies being explored.

Many aspects of the control plane have been explored. Some works focus on traditional controller-based solutions [2, 3], while others propose work centric architectures, as seen in [4]. From an operating system perspective, there are studies on virtual machines [1] and containers [7], which focus on the overhead that these technologies introduce and how they could be less significant.

OpenWhisky [3] and Dirigent [2] are considered distinct at the a fundamental level which means that both architectures are based on different design principles. OpenWhisky uses a queue to manage the requests and a reactive database to deliver responses, following an event based design. Dirigent relies on direct calls between components to handle requests focusing mainly on an in-memory data storage and using a database to keep state information, which resembles a conventional microservice architecture.

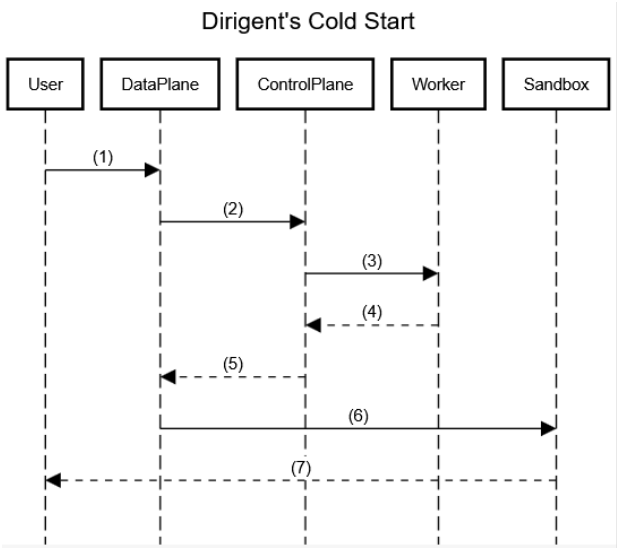

Figure 1. The request path starts when an user invokes a function (1). The Data Plane checks resources with the Control Plane (2), which demands a Worker to create a sandbox (3). Then, the Worker confirms that the sandbox is ready (4), so the Control Plane tells Data Plane to proceed (5). The Data Plane sends the invocation (6), and the Sandbox return it to the user (7).

Although both architectures are reliable and effective to some extent in their own ways, they fail to address a main problem in modern FAAS architectures: The responsibility of executing actions. In most control planes the responsibility for resource instantiation is shared among components, rather than being delegated to the most appropriate actors. This creates a complex communication schema, because the information to execute a task is not located where it will be executed. As a result, to execute something the components always need to request the information to other actors instead of instead of directly executing the action. This creates inefficient and complicated request paths that can be seen in the Dirigent architecture (1), which despite being state-of-the-art, lacks a clear and direct request path.

Therefore, we propose Mimir, a control plane for Serverless functions based on local decisions and responsibility delegation. Mimir uses an aware data plane design allowing workers to schedule their own runtimes. It also ensures that the user's function invocation reaches the running-environment as fast as possible through simplified hot and cold paths. Furthermore, Mimir considers a shared network that will reduce the cold start time for a given user's function using user realms. These architectural choices can be defined as the following properties which ensures Mimir enhanced performance:

- Data Planes capable of making autonomous function placement decisions
- Local Scheduling Decisions
- Simple communication paths for function invocations

The remainder of this paper is organized as follows: Section (2) provides an overview of Mimir's architecture and explains how its components are structured. Section (3) describes the detailed design of each component. Section (5) analyze the function invocation life cycle. Section (6) discusses the technologies and design choices used to build Mimir. Section (8) compare Mimir and Dirigent in terms of communication efficiency and latency. Finally, Section (9) outline directions for future work and Section (10) conclude by summarizing the main findings of this study.

2 Architecture Overview

The Mimir architecture (2) is designed around the principle of functional autonomy, distributing the decisions across three main components: the Controller, the Data Plane, and the Workers. While the system utilizes a centralized Database for state persistence and a Queue System for asynchronous tasks (see 6.4), the critical path for function invocation is optimized through direct communication and local state awareness.

The Controller is responsible for the function data, components registrations and the worker health information. The objective is to keep the system aware of any components failures and also to maintain the data planes informed about the workers resource usage (see 3.1). The controller uses the database to keeps its state, thus in case of any failure it can reconstruct its last state know by the database.

Data Plane(s) in Mimir's architecture, mostly acts as a resource aware load balancer. It receives information about the workers state and uses this data to define what worker or user realm should handle such request (see 3.2).

Workers are responsible to schedule resources (here represented as users realms, see 6.2) and inform their availability to the controller, so then it can inform the data planes about the resource usage and available endpoints (see 3.3).

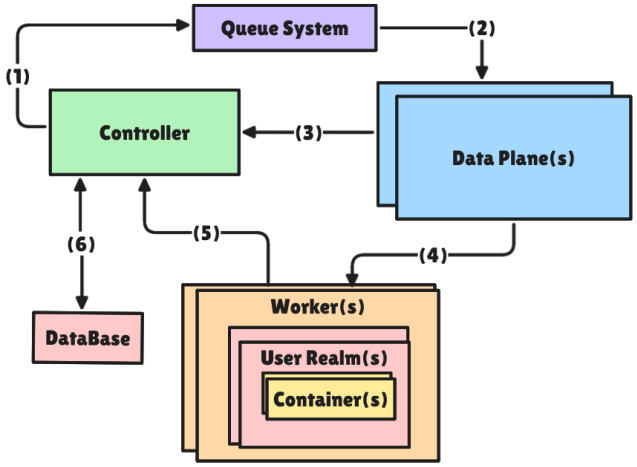

Figure 2. Mimir Architecture Description - The controller publishes information through the queue system (1). All Data Planes then consume this information (2). Each Data Plane registers itself through a GRPC call (3). The Same happens for all workers (5). Each Data Plane, when receiving a request for a non provisioned function send it to a worker or directly to an user-realm (4). The database is used to keeps the controller state (6).

3 Mimir's Design

Mimir's Design is based on local decisions from each component, workers are responsible for scheduling resources, and all data planes are kept aware of worker resource usage.

3.1 Controller

The Mimir controller manages the de-registration and registration of components and functions, as well as the distribution of resource information.

When a data plane is alive, it keeps sending heart beats to the controller, which registers the data plane. But if these heart beats stop, the controller considers that the data plane is no longer alive. The equivalent happens with workers, but it is registered first and then when in functioning it sends heart beats to the controller, in addition these heart beats messages contains the CPU and Memory usage of each worker. When workers register themselves the controller informs all data planes using the queue system, the equivalent happens when a worker is not available anymore, i.e., it stops sending heart beats and then it is de-registered from all data planes.

Functions (de)registrations are triggered by users, who can register a function and its image. When it is registered, the function information will be transmitted for all data planes thought the GRPC's calls.

The controller's state consists of information about registered workers and functions. This data is stored in the

database, so the controller can recover its state after a failure (see 6.1).

3.2 Data Plane

All Mimir data planes are responsible for keeping track of function registrations, performing load balancing, and maintaining information about worker resource allocation endpoints (see 6.2).

All data planes are replicas with eventual consistency on the state information which includes the allocation endpoints from each worker and function information. Consistency is maintained using data from controller and the queue system.

Each data plane is responsible for keeping the registered functions for future user invocation (see 5). This information is recovered from the queue system in a publisher and subscriber strategy: the controller publishes updates, and all data planes consume them.

In a similar manner, the resource usage of each worker is obtained from the queue system. This data allows it to perform a resource aware load balancing (see 6.3).

3.3 Workers

All Mimir's Workers are responsible for the function scheduling. Each worker has an endpoint that is capable of allocating new sandboxes for a function when triggered. This design ensures a more fluid hot and cold path, in such a way that from the data plane's perspective there is no difference between a cold and warm start, disregarding the time of invocation.

To manage resources efficiently, each worker keeps information about user realms and functions, ensuring that a worker contains at most one user realm per user (see 6.2).

4 Mimir's API

Mimir's API is composed of multiple GRPC endpoints, but most are internal; only a few are intended for direct user interaction. For simplicity, this section describes only the user's API. Users can interact with Mimir through three main endpoints: registering a function, deregistering a function (both with the Controller), and invoking a function (with the Data Plane).

The registration and deregistration endpoints allow users to add or remove functions from the Mimir control plane. After one of these endpoints is called, the relevant information is shared among all Data Planes, enabling or preventing the function from being invoked. Both operations require only the function name and the user realm as input.

Function invocation is performed via a GRPC call to the Data Plane. The user provides the function name, which serves as the identifier, and the user realm, which could correspond to a user ID.

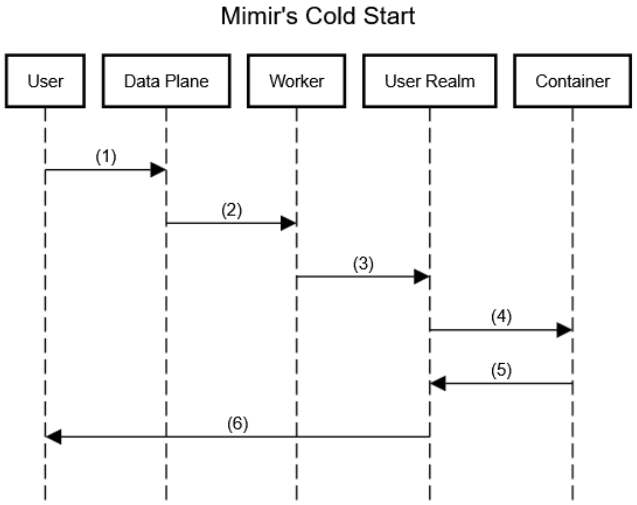

Figure 3. Cold Start sequence diagram - The user trigger the function (1). Then the data plane sends it to the worker with lowest resource usage (2). The worker then creates the user-realms and pass the request up-front. The user-realm creates the sandbox and then executes it (4) and return it (5,6).

5 Function's Invocation Life Time

All invocations first reach the data plane and it has three different methods: cold start, semi-cold start and warm start. A cold start is commonly defined as an invocation without any resource provisioned. Mimir's case is different from other implementations ([2], [4]), since a cold start requires creating a user realm and then allocating a function sandbox. A semi-warm start is realized when the user-realm already exists for the function, but no sandbox is available. A warm start occurs when all resources are ready, so the function can be executed with minimal overhead.

5.1 Cold Start

The Mimir's Cold Start (figure 3) is defined by the complete resource allocation. First, the user calls the data plane, then the data plane tries to find a function sandbox or user-realm. In case of non-existence of both, the request is sent to the worker with the lowest resource usage. The worker creates the user realm (if needed), then the request is forwarded to the user realm, which creates the sandbox, executes the function, and returns the result to the user.

5.2 Semi-Cold Start

A semi-cold start (see Figure 4) happens when the data plane does not know of any sandbox for a function, but a user realm for the user already exists. The user requests the function from the data plane, which redirects the call to the user realm. The user realm creates the sandbox (if needed), executes the function, and returns the result. The advantage of this

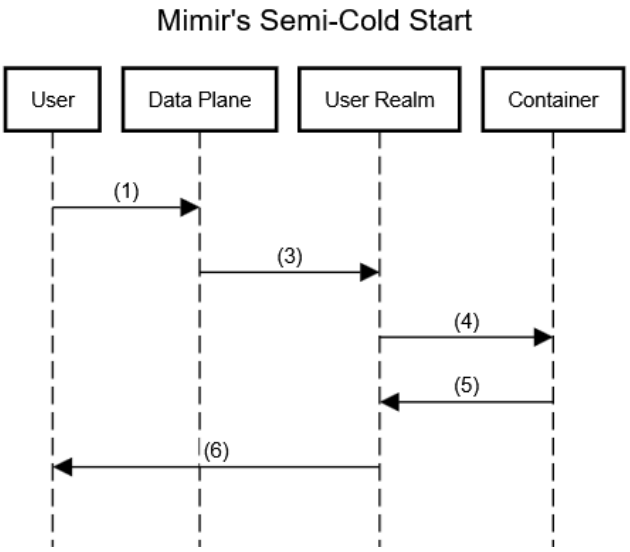

Figure 4. Semi-Cold Start sequence diagram - The user triggers the function (1). The Data Plane sends it directly to the available User Realm (3). The User Realm will provision the sandbox and executes the function (4). Then it is returned to the user (5,6).

method is that the network for the sandbox does not need to be provisioned again (see 6.2), because the user realm would already have a virtual network interface. This would reduce the network setup cost, giving Mimir an advantage over other control planes, which must allocate network resources for each container cold start ([2], [4]).

5.3 Warm Start

The warm start is defined by the absence of resource allocation, in such way that the user requests the function execution and Mimir is able to redirect the request directly to a sandbox. In this scenario almost no overhead would be added. The process is similar to the semi-cold start (see Figure 4), but no sandbox needs to be provisioned.

6 Implementation

Mimir was implemented using Go [5], considering its built-in resources for distributed and parallel systems. Dirigent [2], a control plane for serverless functions, was used as a reference for Mimir's development, as it serves as a comparison point in this work. Concurrency was addressed using Go lock systems which allows the use of shared read locks, creating conflicts only when writes concurred with reads.

6.1 Controller

The Controller is designed to have as little responsibility for scheduling and resource management as possible. It only receives registrations from Data Planes and Workers through GRPC connections. Once connected, each main component of Mimir is expected to send heartbeat messages to the controller. The controller then informs all data planes about the current state using the Nats Queue System [6].

The controller stores its state in a Redis database. If restarted, it can recover its state from this storage. Users can interact with the controller through a GRPC API to add or remove functions from Mimir. Each function is identified by its image and name.

6.2 Workers

Mimir's workers keep an in-memory, thread-safe structure of user realms and their functions. Each user can have at most one realm, so all their functions are grouped in the same environment and share the same virtual network device. If a worker fails, it stops sending heartbeats to the controller, which then deregisters it from all Data Planes as a resource provider.

Each worker provides a resource provisioning endpoint, available through a reverse proxy implemented with Go's `net/http` package. This allows data planes to request sandbox provisioning without knowing the internal details. From the perspective of any Mimir Data Plane, this endpoint behaves like any other endpoint, creating a decoupled relationship between workers and data planes.

An important clarification about Mimir is that **no user-realm internal detail is indeed implemented**, since it is considered that VM's handling, scheduling and eviction mechanisms are out of scope for this work.

6.3 Mimir's Load Balancer

The load balancer mainly uses worker resource usage, but also considers the availability of user realms and containers for a given function. If a function endpoint is available, the request is sent there. If not, but a user realm is available, the request is sent to the user realm. If neither is available, a provisioning endpoint is selected. In all cases, the worker with the lowest resource usage is chosen.

6.4 Queue System

The publisher/subscriber framework used in Mimir is the Nats Queue System, which provides two main methods of message distribution: ephemeral message keeping and long-term message keeping based on storage limits.

Ephemeral message keeping ensures that a message remains in memory until all consumers acknowledge its delivery. In Mimir, this mechanism is used for function endpoints and worker health information, since these types of data are not part of the persistent state of the controller or data planes.

Long-term message keeping ensures that messages are only deleted when storage space is no longer available. This means that any new consumer will receive all messages in such a queue. Mimir uses this method to distribute state information, such as function data and worker endpoint

provisioning. Both types of information can be added and deleted using this queue system

7 Related Work

Several control plane architectures have been proposed to address the challenges of scheduling and resource management in FAAS environments. The relevant for this work are OpenWhisky and Dirigent.

OpenWhisky [3] adopts an event based design, using a queue to manage incoming requests and a reactive database to deliver responses. This architecture enables efficient handling of asynchronous workloads and decouples components for better scalability. However, relying on a central queue and database can lead to complex communication paths and potential latency overheads.

Dirigent [2] represents a more traditional control plane, relying on direct calls between components and maintaining most state information in memory, with a database used only for persistence. This design resembles a microservices architecture and aims to minimize external dependencies. While Dirigent achieves a reasonable latency in warm and cold requests its paths can become complex, and the shared responsibility for resource instantiation can lead to inefficiencies.

In comparison with these previous approaches, Mimir focuses on simplifying the communication schema by delegating scheduling and resource management responsibilities directly to the workers. By reducing the number of components involved in the critical path and allowing local decision making, Mimir aims to minimize latency, as demonstrated in the experimental evaluation

8 Experiments

The experiments were conducted to compare the cold start and warm start performance of Mimir and Dirigent. Tests were run on three Google Cloud machines (e2-standard-4), each with 4 vCPUs and 16 GB RAM. Each machine hosted one component (Controller, Data Plane, or Worker) of either Mimir or Dirigent. This setup was chosen to better simulate real network traffic and to create a more deterministic and easily replicable environment.

An important detail is that Dirigent was modified to ignore the downscaling process during these experiments. Otherwise, this subprocess would consume machine resources, which would be unfair to Dirigent, especially since Mimir does not have this mechanism and would not experience the same workload.

These experiments focus on two main questions:

- Is there any benefit in communication time when the appropriate actors execute actions in a cold start, beyond the simplification of the request path?
- Is there any performance loss in a warm start due to Mimir's communication scheme?

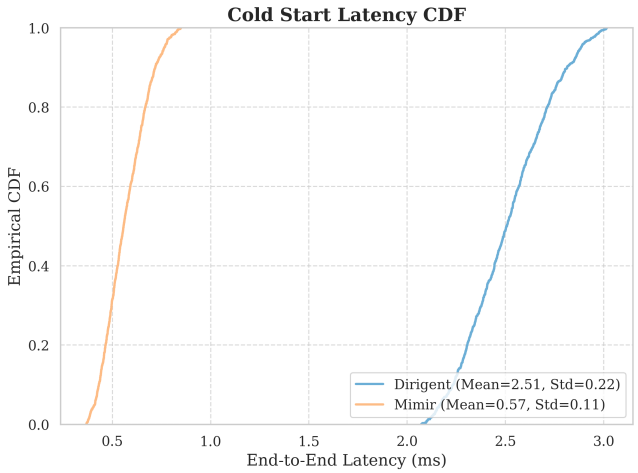

Figure 5. Comparison between Dirigent and Mimir in a cold start scenario. Dirigent has a higher latency and variation (2.51 ms and 0.22 ms), while Mimir shows much lower latency and variation (0.57 ms and 0.11 ms).

Taking these two questions into account, we designed two types of experiments: Cold Start and Warm Start. For each scenario, we performed 1,000 executions with both Mimir and Dirigent, measuring their performance under both conditions.

8.1 Cold Start

The goal of this experiment was to measure the benefits of removing a complex communication scheme and delegating responsibility to the right actors. We measured the latency from when a request is received by the Data Plane to when the response is returned to the user, in both Mimir and Dirigent, during a cold start scenario that activates every component of the control plane.

The results (5) show that Mimir is at least four times faster on average compared to Dirigent in the same scenario. Mimir's lower latency is due to its simplified request path, which involves fewer components and results in a shorter, faster path. Furthermore, Dirigent shows about twice the variation in latency. This is because each internal GRPC call requires computational/network resources and, while efficient, introduces more unpredictability compared to Mimir, since it does considerably less requests in this cold start request path.

8.2 Warm Start

The goal of this experiment was to measure the latency of a warm start in both Mimir and Dirigent. We measured the time taken for a request to reach the Data Plane and be passed to the correct sandbox for execution. Unlike the cold start experiment, in a warm start only the Data Plane, user realm (for Mimir), and sandbox are involved.

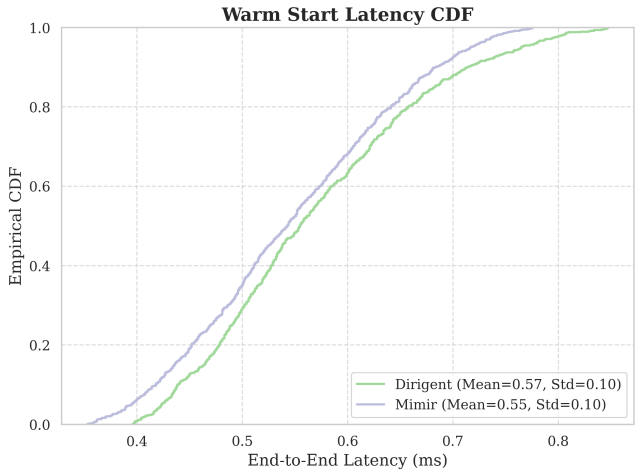

Figure 6. Comparison between Dirigent and Mimir in a warm start scenario. Dirigent has a latency and variation of 0.57 ms and 0.10 ms. Mimir has a latency and variation of 0.55 ms and 0.10 ms.

Both Mimir and Dirigent show similar performance in the warm start scenario (6), with average latencies around 0.55 ms and 0.57 ms, respectively, and very small variation in both cases. The results are close because, in a warm start, both systems use almost the same request path and technologies, such as proxies and GRPC calls.

However, Mimir is slightly better due to the lock system used by Dirigent, which locks the endpoint and prevents further requests when it is being upscaled or analyzed for upscaling. This causes some competition for endpoints between the request path and the upscaling process in Dirigent. Mimir maintains almost the same latency and variation as in the cold start, since the request path is basically the same, with the only difference that the worker is also involved in the cold start.

9 Future Work

The Function as a Service (FAAS) environment remains a rich area for research, as highlighted by [4]. For this reason, we believe there are still many opportunities to further develop and improve upon this work.

One important direction is the development of user realms using a shared network system, which would be fundamental for achieving greater usability. Implementing user realms with Firecracker VMs [1] would provide better isolation than simple containers and enable a more robust environment for shared networking.

Another challenge is enhancing the load balancing mechanism to better handle request concurrency. Since there is no single component, such as the controller, with full control, it is consider that coordination among all Data Planes is required even if indirect. Introducing throttlers in each Data Plane would help prevent overloading underutilized workers. A request throttler would limit the number of requests sent directly to workers or user realms, even if they appear to have available resources, thus ensuring fair and efficient resource usage.

Similar coordination is needed for eviction policies. To evict a sandbox, the system must have some knowledge or prediction about future requests and the state of resources across all workers. The key question is when and how to evict a sandbox, considering both upcoming requests and the unknown state of other workers.

Finally, further exploration of the Nats Queue System for state distribution could be valuable. Since much of the controller's work involves distributing and managing the state of other components, using the queue to store and distribute state information could enable efficient component state reconstruction by reading from topics, rather than relying on a large number of GRPC calls as in Dirigent [2]. This would create an environment where a Database for state keeping would be useless.

10 Conclusion

This work introduced Mimir, a control plane for serverless functions that emphasizes local decision-making and responsibility delegation to improve performance and simplify communication paths. By distributing autonomy among the Controller, Data Plane, and Workers, Mimir addresses key limitations found in previous architectures such as Dirigent and OpenWhisky, particularly regarding complex request paths and shared responsibility for resource management.

Mimir shows that a control plane designed around local autonomy and clear responsibility can deliver tangible benefits in serverless environments.

A Network Sharing

The concept of network sharing (7) was, during the development of this work, mostly theoretical and not fully explored in practice. However, a simple equation was developed to describe the balance between the number of functions required in a user realm and its shared network system, showing when this approach becomes more efficient than using individual sandboxes with separate networks.

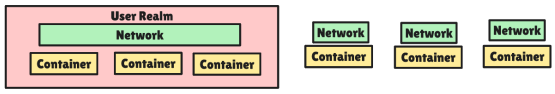

Figure 7. Network Sharing between three sandboxes and an user realm in comparison with simple sandboxes use.

This equation can be directly derived from the relation (1):

$$(n_t + c_t) \times h = (v_t + n_t) \times u + c_t \times h \tag{1}$$
$$n_t \times h = (v_t + n_t) \times u$$
$$\overline{f} = \frac{v_t}{n_t} + 1$$

where:

- v_t: Average time to initialize a User Realm VM.
- c_t: Average time to initialize a container.
- n_t: Average time to create a virtual network device.
- $\overline{f}$: Average number of functions.
- h: Total functions ($h = \overline{f} \times u$, where u is number of realms).

This leads to the inequality (2), which describes the inflection point where user realms become more efficient for cold starts. That is, if on average there are more than $\frac{v_t}{n_t} + 1$ functions per user realm, then using user realms with shared networks is more efficient than using individual networks for each sandbox:

$$\overline{f} > \frac{v_t}{n_t} + 1 \tag{2}$$

References

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 419–434. https://www.usenix.org/conference/nsdi20/presentation/agache

[2] Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, and Ana Klimovic. 2024. Dirigent: Lightweight Serverless Orchestration *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 369–384. doi:10.1145/3694715.3695966

[3] The Apache Software Foundation. 2016. Apache OpenWhisk. https://openwhisk.apache.org/. Accessed: 2026-03-28.

[4] Alexander Fuerst, Abdul Rehman, and Prateek Sharma. 2023. Ilúvatar: A Fast Control Plane for Serverless Computing. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (Orlando, FL, USA) *(HPDC '23)*. Association for Computing Machinery, New York, NY, USA, 267–280. doi:10.1145/3588195.3592995

[5] Robert Griesemer, Rob Pike, and Ken Thompson. 2009. The Go Programming Language. https://golang.org. Accessed: 2026-02-17.

[6] NATS Authors. 2026. NATS: Cloud Native, Open Source, High-performance Messaging. https://nats.io/. Accessed: 2026-02-17. Open-source messaging system supporting publish/subscribe and more. Source: nats.io..

[7] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 57–70. https://www.usenix.org/conference/atc18/presentation/oakes